SYSTEM DESIGN · INTERVIEW DEMO

Retrofit Copilot: trustworthy AI for energy retrofit decisions

An energy certificate goes in. A cited, calculated retrofit plan comes out. This page explains how the system is built, how the stack fits together, and why each design choice was made over its alternatives.

3 AI features: extract, plan, ask	**3** fallback levels per feature	**5** evaluation suites, 2 gated in CI offline	**21** tests, no API key needed
0 costs, grants or CO_2 figures from the LLM			

01 · FRAMING

Goals and constraints

Wallround's numbers end up in investment decisions and bank-ready reports, so a wrong figure costs money and trust. I designed for that risk first and for cleverness second.

What the system must do

- Read a certificate from a text PDF, a scan or a phone photo.
- Let a person check and correct every value before it is used.
- Propose measures with costs, grants, CO_2 and rent effect, and cite the funding rules behind them.
- Answer funding questions from sources, or say it does not know.

Quality goals, in priority order

1. **Auditability.** Every number can be traced to a table or a formula.
2. **Graceful degradation.** No single failure stops the user.
3. **Measurable quality.** Changes to prompts or models are scored before they ship.
4. **Cost and latency control.** Every call is priced and timed.
5. **Reuse.** The next AI feature should take days, not weeks.

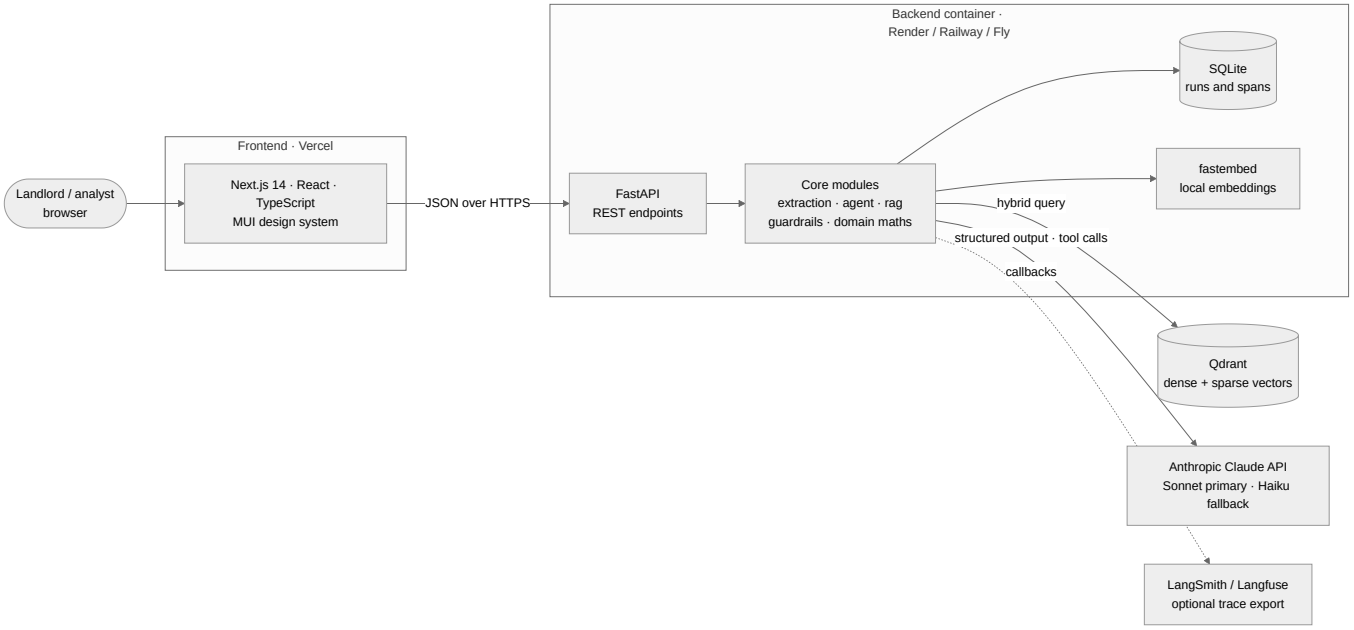
Explicit non-goals for the demo

Authentication, multi-tenancy, horizontal scaling, and legal-grade funding advice. These are listed in Path to production, so the scope is a decision rather than an oversight.

02 · ARCHITECTURE

System context

A modular monolith: one FastAPI service with clear internal boundaries, a Next.js client, one vector store, and the Claude API. Each box below is a separate deployable or a third-party service.



If Qdrant is unreachable the backend switches to an in-memory Qdrant and re-indexes the knowledge base. If the embedding model cannot download it uses hash embeddings. If Claude is unavailable each feature has a rule-based path. The system still works when any one of the right-hand services is down.

03 · TECHNOLOGY

The stack, layer by layer

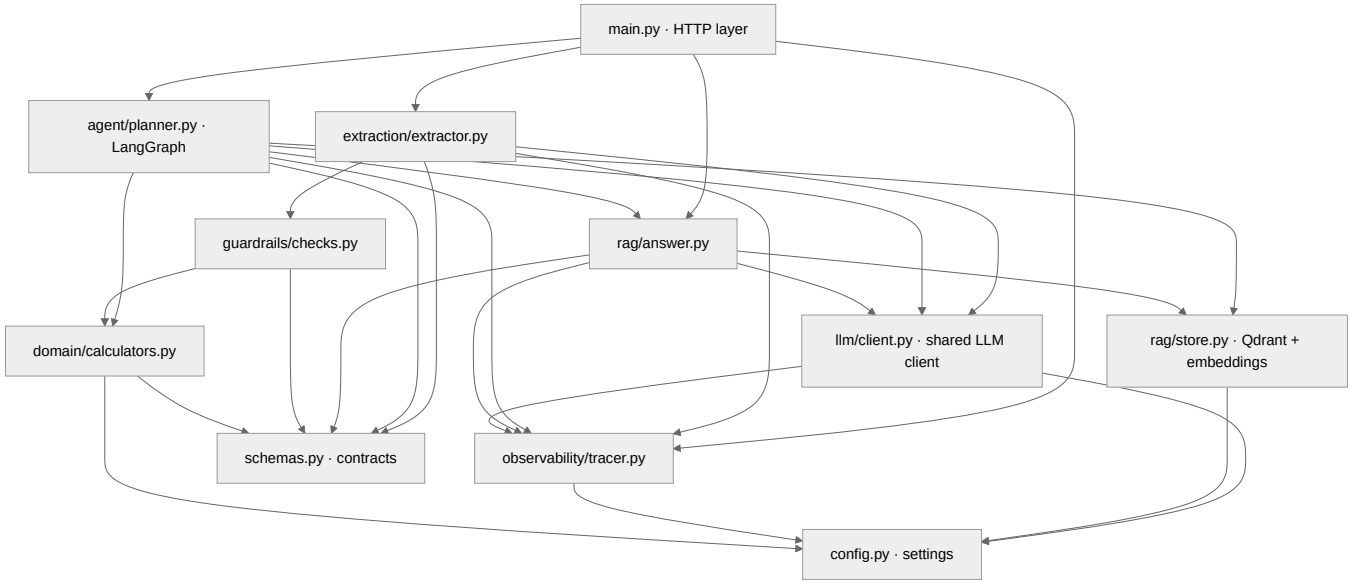
Each layer talks only to the one below it through typed contracts. The Pydantic schemas in `schemas.py` mirror the TypeScript types in `lib/api.ts`, so a contract change breaks loudly on both sides.

EXPERIENCE	`Next.js 14 App Router` `React 18` `TypeScript 5.7` `MUI 6 + Emotion` `next/font`
API	`FastAPI` `Uvicorn` `Pydantic 2` `pydantic-settings + dotenv`
AI ORCHESTRATION	`LangGraph state graph` `LangChain Core runnables` `with_fallbacks · with_structured_output` `versioned prompt files`
MODELS	`Claude Sonnet 5.5 (primary)` `Claude Haiku 5.5 (fallback)` `vision for scans and photos` `LLM-as-judge for evals`
RETRIEVAL	`Qdrant 1.13` `fastembed multilingual MiniLM` `BM25-style sparse vectors` `RRF fusion`
DOMAIN LOGIC	`calculators.py` `reference JSON: costs, BEG caps, CO₂ factors, classes` `guardrail rules`
DOCUMENTS	`pypdf (text layer)` `pypdfium2 + Pillow + reportlab (samples)`
QUALITY AND OPS	`pytest` `ruff` `eval harness + thresholds` `GitHub Actions` `SQLite tracer` `LangSmith` `Docker Compose`

CHOICE	WHY IT FITS HERE	MAIN ALTERNATIVES	WHAT I GAVE UP
Python backend, TypeScript frontend	The AI and document ecosystem (LangGraph, Qdrant client, fastembed, pypdf, evaluation tools) is strongest in Python. The posting accepts TypeScript and/or Python.	All TypeScript with Next.js API routes and the Vercel AI SDK	One language across the stack; shared types need to be kept in sync by hand

CHOICE	WHY IT FITS HERE	MAIN ALTERNATIVES	WHAT I GAVE UP
FastAPI	Pydantic models are the API contract and the LLM output schema at the same time. OpenAPI docs come free at `/docs`.	Django, Flask, Litestar	Batteries such as auth and admin
LangGraph	The agent loop is an explicit, testable state machine with an iteration cap, and model calls can be injected in tests.	Prebuilt ReAct agent, plain while-loop, LlamaIndex agents, Claude Agent SDK	An extra dependency and its upgrade churn
Claude with a fallback model	Strong document vision, reliable JSON-schema output and tool use. Haiku is cheaper and fast enough as a backup.	OpenAI or Gemini as a second provider	Provider diversity; a regional outage hits both models
Qdrant	Native named dense and sparse vectors with server-side IDF and RRF fusion in one query. Runs in-memory for tests.	pgvector, Pinecone, Weaviate, FAISS	Another stateful service to run, compared with pgvector inside an existing Postgres
fastembed local embeddings	No per-call cost, no data leaves the server, multilingual for German and English.	Voyage, OpenAI or Cohere embedding APIs	Some retrieval quality, plus a model download and about 1 GB RAM
MUI design system	Accessible components, theming and a fast path to a polished product UI.	Tailwind + shadcn/ui, plain CSS	Bundle size and Emotion's runtime styling cost
Own tracer + LangSmith	Our own metrics are what CI and the dashboard gate on. LangSmith adds step-by-step debugging views.	Vendor only (LangSmith, Langfuse, Datadog LLM) or OpenTelemetry	Some duplicate data

How the backend modules depend on each other

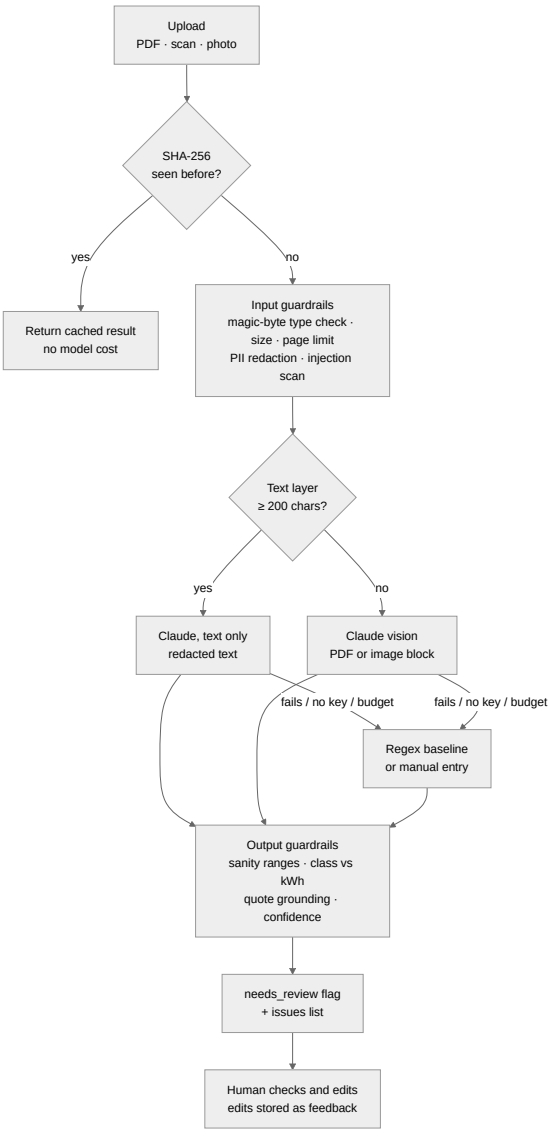


Dependencies point one way: features at the top, shared infrastructure in the middle, contracts and configuration at the bottom. Nothing in `domain` or `guardrails` knows about LLMs, so the rules can be unit tested without mocks.

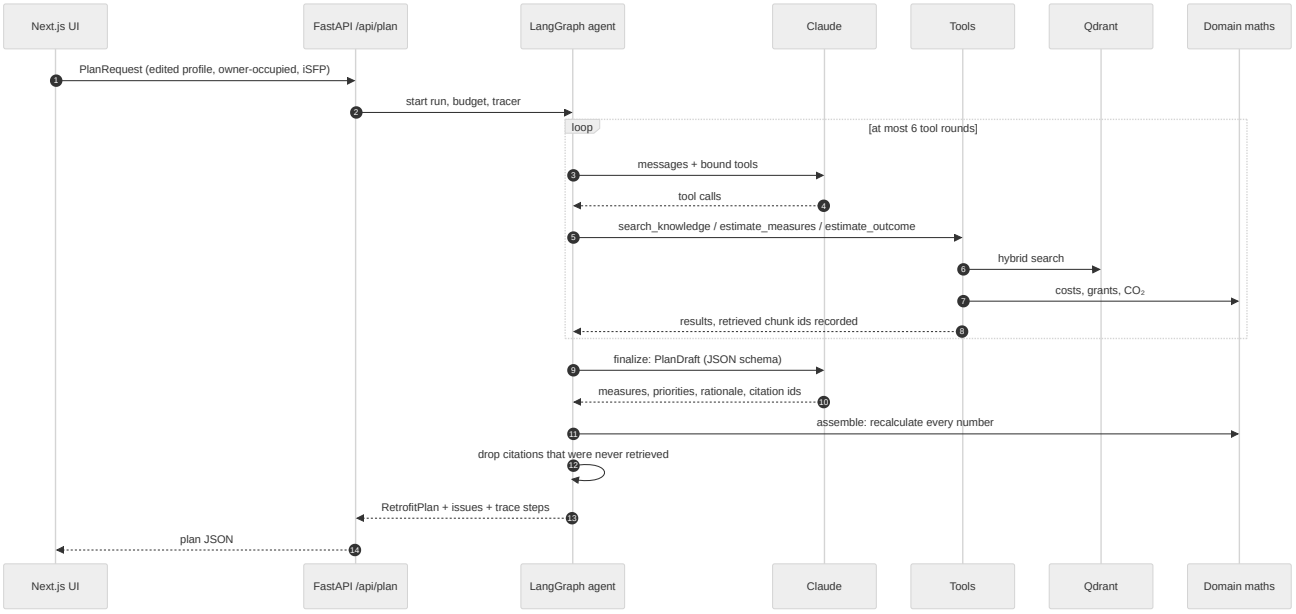
04 · RUNTIME

Request flows

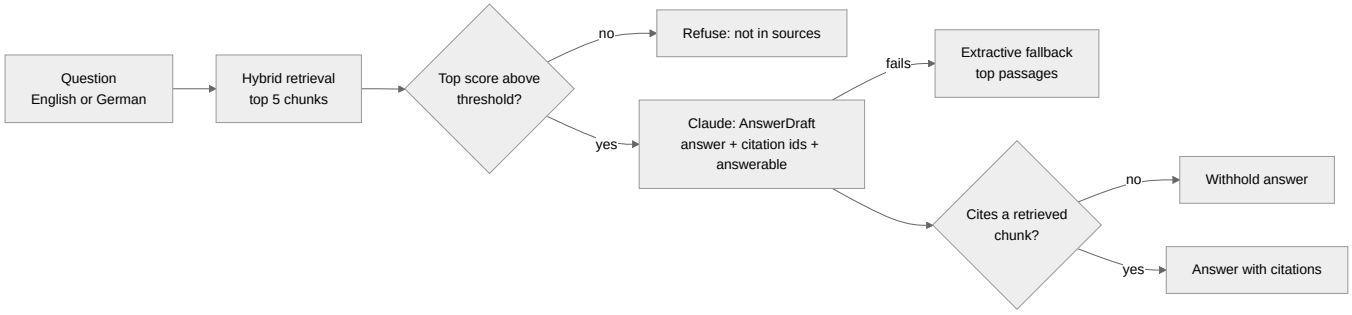
Extraction: certificate to building profile



Planning: profile to cited plan

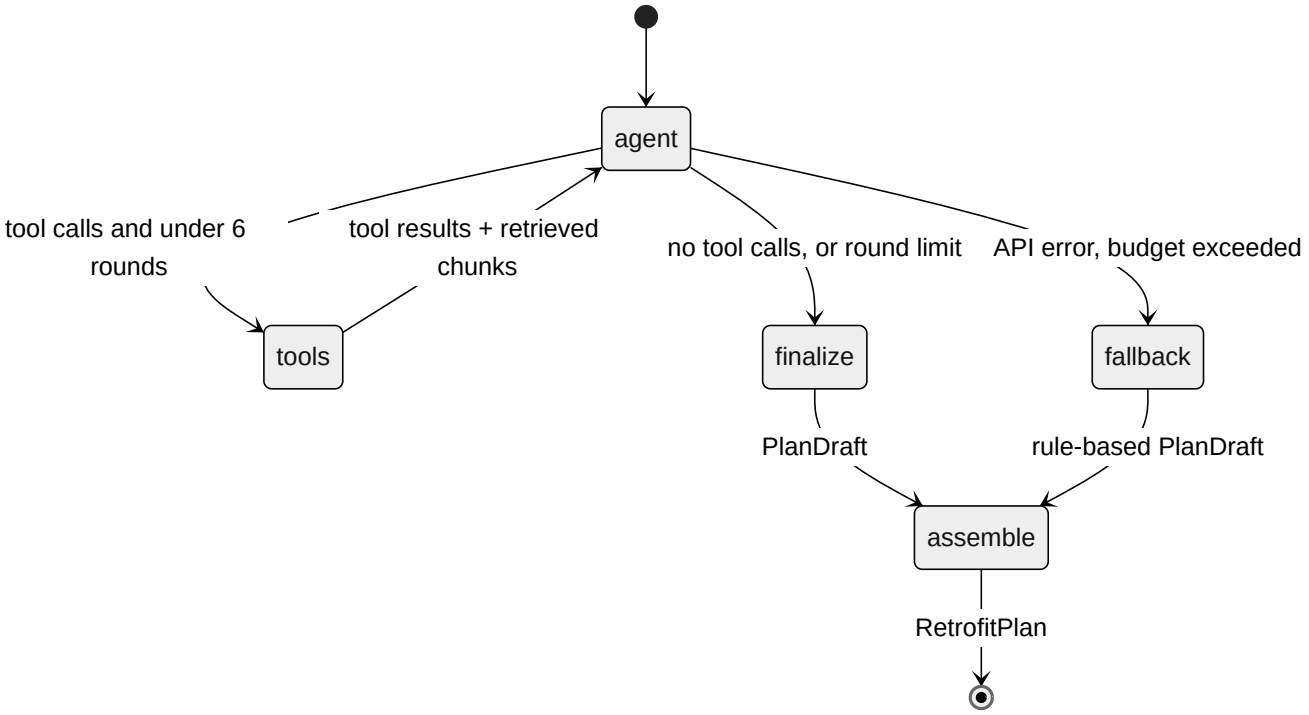


Ask: grounded question answering



05 · AGENT DESIGN

The agent decides what; code calculates how much



State

Messages, retrieved chunks (append-only), iteration count, and the final draft. Retrieved chunks are the evidence that citations are checked against.

Tools

`search_knowledge`, `estimate_measures`, `estimate_outcome`. The tools are closures over the request, so the model cannot change the building data.

Finalize

A separate structured call returns only choices, priorities, rationale and citation ids. It has no fields for money, so the schema itself prevents invented numbers.

Why this split matters

LLMs are unreliable at arithmetic, and their numbers cannot be audited. The grant rules alone have caps per dwelling unit, a bonus that only applies above a threshold, and different programmes for the envelope and for heating. That logic lives in tested Python, and the model only reasons about which measures make sense.

Example: grant caps the code must get right

Eligible cost per building and year under BEG since 21 July 2026, by number of dwelling units. The 8-unit values (€121,000 and €240,000) match the official reform example and are a unit test.



BAFA envelope, with iSFP

BAFA envelope, without iSFP

KfW heating (dashed; almost the same caps as BAFA without iSFP)

06 · RETRIEVAL

Hybrid retrieval with verified citations

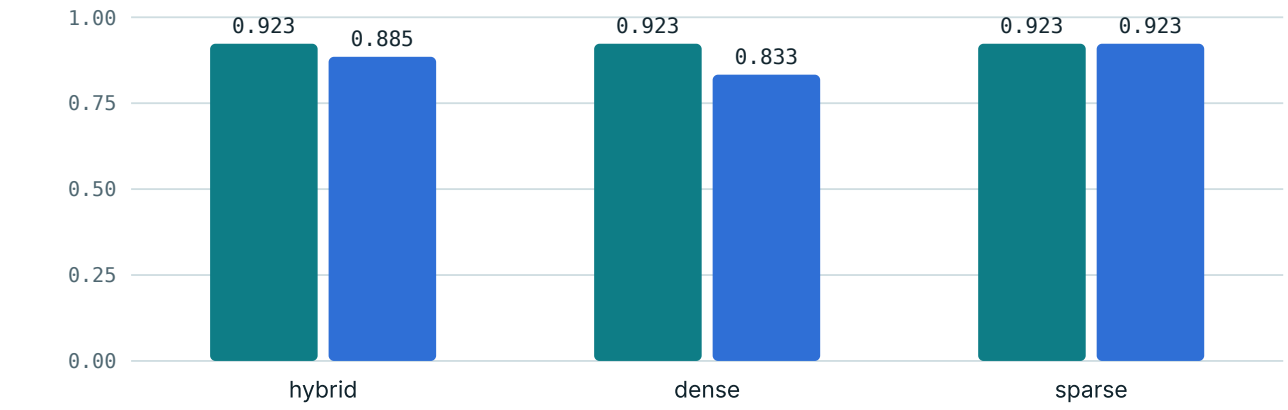
Indexing

- Markdown knowledge base, split by heading, then into chunks of about 900 characters at paragraph boundaries.
- Stable ids such as `beg-em-envelope#2`, so citations survive re-indexing.
- Each chunk gets a dense vector (multilingual MiniLM) and a sparse vector (term frequency; Qdrant applies IDF on the server).
- The knowledge base version is a hash of its files and is stored with every chunk.

Querying

- Two prefetches of 20 candidates each, dense and sparse, fused with Reciprocal Rank Fusion into the top 5.
- Dense catches paraphrases across languages ("Zuschuss Dämmung" and "insulation grant").
- Sparse catches exact identifiers that embeddings blur: "iSFP", "KfW 458", "§ 559".
- RRF needs no score calibration between the two very different scales.

Offline retrieval eval, 13 labelled questions



Measured with the offline hash embeddings used in CI, which is why dense search scores lowest here. With the real multilingual model, the trade-offs between modes change, and the same command reports them: `make evals`. The point of the chart is that the comparison is measured, not assumed.

07 · RELIABILITY

Failure modes and how each is handled

Three levels of fallback. API level: SDK retries with backoff, a timeout, then the Haiku fallback model. Feature level: a rule-based path that needs no AI. Budget level: a per-request cost cap that skips straight to the rule-based path.

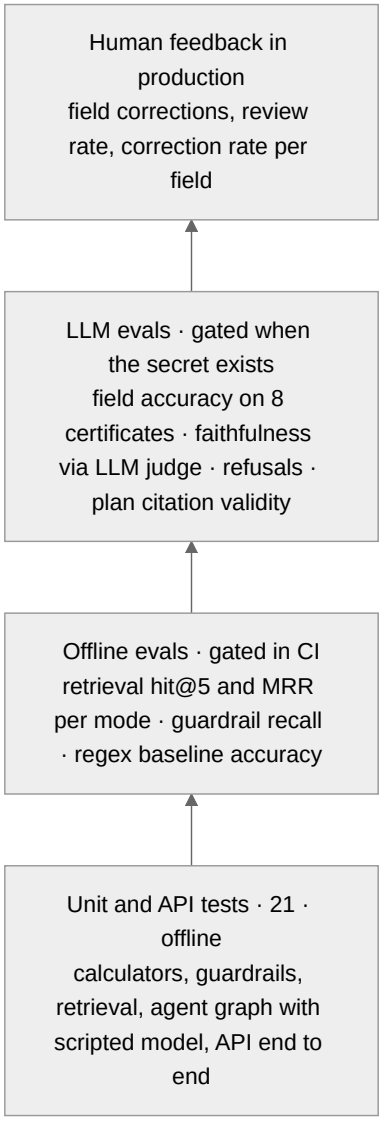
FAILURE	DETECTED BY	RESPONSE	USER SEES
429 or 5xx from Claude	Anthropic SDK	2 retries with backoff, then fallback model	`normal result` plus a "fallback model" chip
Malformed structured output	JSON schema parser	Fallback model; then rule-based path	`review`
Run exceeds cost budget	Telemetry callback raises `BudgetExceeded`	Not in `FALLBACK_ON`, so it skips the second model and goes straight to the rules	`rule-based plan`

FAILURE	DETECTED BY	RESPONSE	USER SEES
No API key	Shared client	Regex extraction, rule-based planner, extractive answers	`header shows fallbacks only`
Scan without text layer	Text length check	Vision path; confidence capped at 0.8	`review likely`
Invented value	Quote not found in source text	Field flagged `ungrounded`	`field highlighted`
Prompt injection in document	Pattern scan + document wrapped as data	Instruction ignored; issue raised	`warning`
Inconsistent certificate	Class versus kWh rule	Issue raised, values kept as printed	`warning`
Citation the agent never retrieved	Assemble step	Citation removed	`warning`
Agent loops	Iteration counter + recursion limit	Forced to finalize after 6 rounds	`normal result`
Tool raises an error	Tool node	Error returned to the model as a tool message	`model recovers`
Qdrant down or embeddings unavailable	Startup probe	In-memory Qdrant; hash embeddings	`works`, lower search quality
Wrong file type or oversized file	Magic bytes, size, page count	Rejected with HTTP 422	`clear error`

08 · QUALITY

Evaluation strategy

Four layers, each catching a different class of regression. The lower layers are cheap and run on every push; the upper layers cost money and run when a model or prompt changes.



Gates

`evals/thresholds.json` sets a minimum per metric, for example hit@5 ≥ 0.85, field accuracy ≥ 0.9 and faithfulness ≥ 4.0. `--gate` exits non-zero below any of them, which fails the GitHub Actions job.

Known weaknesses

Small datasets (8 certificates, 14 questions). An LLM judge from the same model family can share blind spots. The next step is to grow the set from user corrections and add a second, independent judge or human spot checks.

09 · OPERATIONS

Observability

Data model

Run: one API request. Kind, duration, status, total cost and tokens, LLM call count, fallback used, needs review.
Span: one step. Kind (llm, tool, retrieval, guardrail, fallback, cache), model, tokens, cost, latency, error, prompt version, tool calls.

Metrics derived

Per feature and per model: p50 and p95 latency, error rate, fallback rate, review rate, average cost and tokens. Human corrections per field. Exposed at `/api/metrics` and on the Observability page.

The same LangChain callbacks also feed LangSmith when its environment variables are set, which gives a click-through view of every agent step. In production I would export spans with OpenTelemetry to the team's existing stack and alert on p95 latency, error rate, fallback rate and daily spend.

10 · CODEBASE

Code structure and what each part does

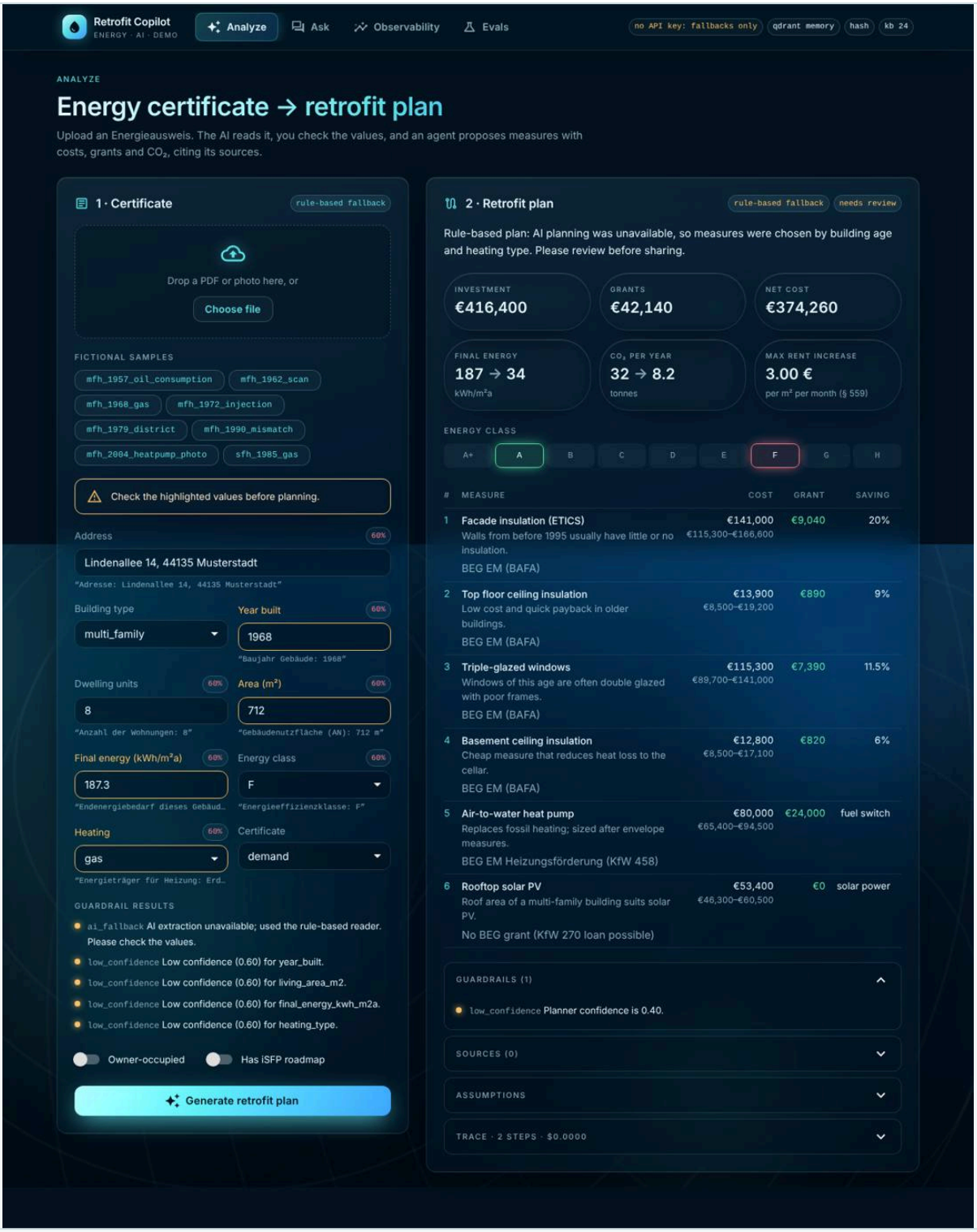
backend/app/	The Python service. Features on top, shared infrastructure below.
main.py	HTTP layer: health, extract, sample extract, plan, ask, feedback, runs, metrics, latest eval. Opens a traced run per request.
config.py	Typed settings from environment and `.env`; also exports `.env` so LangSmith and the SDK see it.
schemas.py	Pydantic contracts: BuildingProfile, PlanDraft, RetrofitPlan, AnswerDraft, Issue. Used as API types and as LLM output schemas.
llm/client.py	Shared LLM client: model setup, timeout, retries, primary-to-fallback chain, JSON-schema output, tool binding, prompt loader with versions.
llm/pricing.py	Token prices per model family for cost tracking.
extraction/extractor.py	Certificate to profile: cache, input guardrails, text or vision path, regex baseline, output guardrails, review flag.
guardrails/checks.py	File sniffing, size, PII redaction, injection detection, sanity rules, class consistency, fuzzy quote grounding, confidence rules.
rag/store.py	Chunking, dense and sparse vectors, Qdrant collection, hybrid search with RRF, in-memory and hash fallbacks.
rag/answer.py	Grounded Q&A: retrieve, threshold, structured answer, citation guardrail, extractive fallback.

`agent/planner.py`	LangGraph graph (agent, tools, finalize), tool definitions, rule-based fallback planner, assemble step that recalculates numbers and checks citations.
`domain/calculators.py`	Deterministic maths: measure costs, heat pump sizing after envelope savings, BEG grants and caps, energy class, CO_2, § 559 rent levy.
`observability/tracer.py`	Run and Span model, LangChain callback for LLM spans, budget check, SQLite store, metrics aggregation, Langfuse hook.
`prompts/*.md`	Versioned prompts: extraction, planner, planner_finalize, answer, judge. The version is attached to every trace.
`backend/evals/`	Datasets (certificates and questions), thresholds, and the runner with five suites and a CI gate.
`backend/tests/`	21 offline tests: calculators, guardrails, retrieval, extraction fallbacks, agent graph with a scripted model, API end to end.
`backend/scripts/`	`make_samples.py` generates the fictional certificates and their labels.
`data/knowledge/`	Seven Markdown documents: BEG envelope and heating, Efficiency House, GEG certificates, CO_2 factors, measures, § 559 levy.
`data/reference/`	One JSON file with cost ranges, savings, grant rates and caps, CO_2 factors, class thresholds and levy rules.
`data/samples/`	Eight fictional certificates: text PDFs, a scan, a phone photo, an injection case and an inconsistent case.
`frontend/app/`	Next.js App Router pages: Analyze (`page.tsx`), Ask, Observability, Evals. `components.tsx` holds shared UI; `theme.ts` and `providers.tsx` set up MUI.
`frontend/lib/api.ts`	Typed API client and TypeScript mirrors of the backend schemas.
`infrastructure`	`docker-compose.yml` (Qdrant, API, web), two Dockerfiles, `Makefile`, `.github/workflows/ci.yml`, `.env.example`.

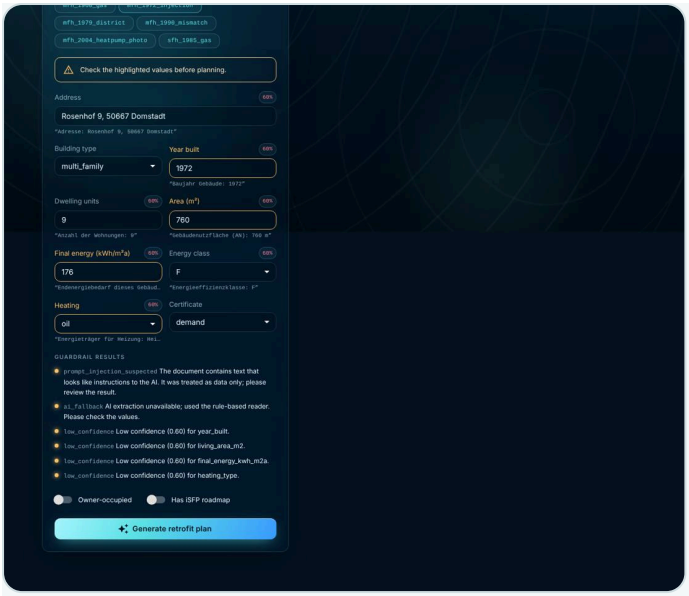
11 · PRODUCT

What the user sees

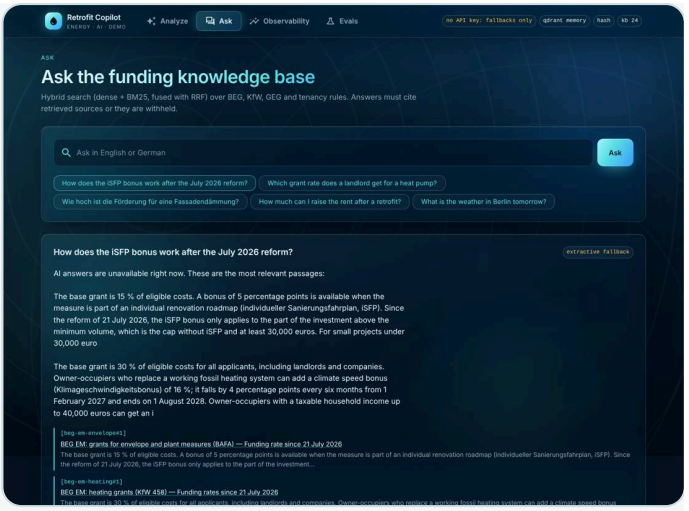
Screens captured without an API key, so they show the rule-based fallback path, clearly labelled as such in the UI. Fonts fell back to system faces in the capture environment.



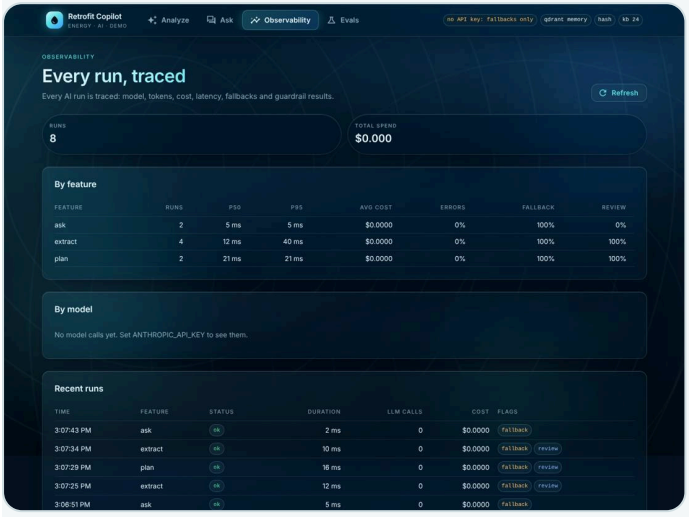
Analyze: extracted fields with confidence and source quotes on the left; the plan with investment, grants, net cost, energy and CO_2 before and after, rent effect and the measure table on the right.



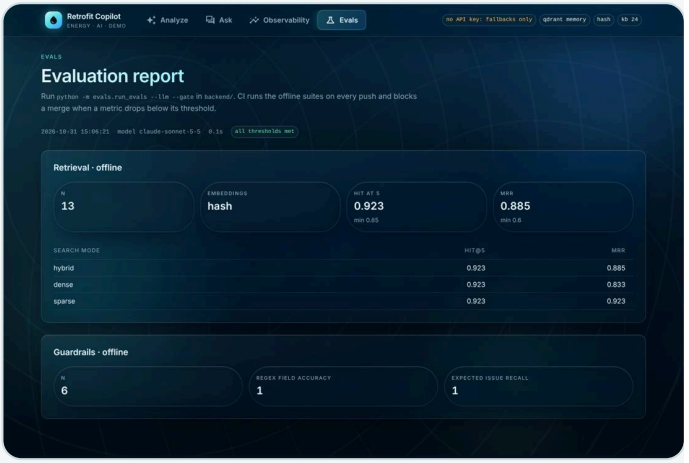
The injection sample: values stay as printed and the guardrail raises a warning.



Ask: answers show the source chunks they are based on.



Observability: latency, cost, fallback and review rates per feature.



Evals: the latest report, compared with the CI thresholds.

12 · DECISION LOG

Key decisions, alternatives and trade-offs

Written as short architecture decision records. In the interview, each one is a ready answer to "why did you do it this way?"

ADR-1 Modular monolith, not microservices

DECISION	One FastAPI service with strict module boundaries and one-way dependencies.
ALTERNATIVES	Separate extraction, retrieval and planning services; serverless functions.
WHY	One team, one deployable, and in-process calls are easier to trace. The module boundaries are where services would split later, for example extraction becoming a queue worker.
TRADE-OFF	Everything scales together; a heavy extraction load affects the planner.

ADR-2 The model decides, code calculates

DECISION	The LLM returns choices, priorities, rationale and citation ids. Python recalculates every figure from reference tables.
ALTERNATIVES	Let the model compute costs; give it a calculator tool and trust its final numbers.
WHY	Numbers must be auditable and identical for identical inputs. The rules are testable and can be updated without prompt changes.
TRADE-OFF	The model cannot reason about cases the tables do not cover; those go to review.

ADR-3 Structured output with JSON schema everywhere

DECISION	Every model output is a Pydantic schema, sent with `method="json_schema"`.
ALTERNATIVES	Free text plus parsing; forced tool calling.
WHY	Typed outputs flow straight into guardrails and the API. Sonnet 5.5 does not support forced tool calling, so JSON schema is the reliable route.
TRADE-OFF	Schemas constrain what the model can express; nuance goes into rationale fields.

ADR-4 Text first, vision only when needed

DECISION	Send redacted text when a PDF has a text layer; send the image only for scans and photos.
ALTERNATIVES	Always vision; a dedicated OCR or document AI service such as Azure Document Intelligence or Textract.
WHY	Cheaper and faster, allows PII redaction, and allows checking every quote against the source.
TRADE-OFF	Two paths to test; vision results cannot be grounded the same way, so their confidence is capped.

ADR-5 An explicit LangGraph graph with injectable models

DECISION	A hand-built graph (agent, tools, finalize) with an iteration cap, rather than a prebuilt ReAct agent.
ALTERNATIVES	Prebuilt agent; a plain loop; LlamaIndex; the Claude Agent SDK.
WHY	Every transition is visible and testable. The tool node records retrieved chunks, which the citation check needs. Tests inject scripted model turns.
TRADE-OFF	More code than a one-line prebuilt agent; framework version churn.

ADR-6 Hybrid search in Qdrant with RRF

DECISION	Dense multilingual embeddings plus sparse term vectors, fused with Reciprocal Rank Fusion in one Qdrant query.
ALTERNATIVES	Dense only; pgvector with Postgres full-text; a cross-encoder reranker such as Cohere Rerank.
WHY	Funding documents mix paraphrased concepts with exact codes. RRF needs no tuning of score scales.
TRADE-OFF	No reranker yet; pgvector would mean one fewer service if the team already runs Postgres.

ADR-7 Fallback ladder, with budget errors excluded

DECISION	Retries, then a cheaper model, then a non-AI path. `BudgetExceeded` is deliberately not in the fallback-model exception list.
ALTERNATIVES	Fail fast with an error; retry forever.
WHY	Users always get a usable, clearly labelled result. Falling back to a second model after a budget breach would spend more money, which defeats the cap.
TRADE-OFF	Fallback results are weaker; they are always marked for review.

ADR-8 Human review before planning

DECISION	Extraction never feeds the planner directly. The user checks the values, and every edit is stored as feedback.
ALTERNATIVES	Fully automatic pipeline; review only when confidence is low.
WHY	Wrong inputs make every downstream number wrong. Corrections are the best source of new eval data.
TRADE-OFF	One extra step for the user; with good accuracy it could become review on exception only.

ADR-9 Own telemetry plus a vendor view

DECISION	A small tracer writes runs and spans to SQLite; LangSmith or Langfuse receive the same events when configured.
ALTERNATIVES	Vendor only; OpenTelemetry from day one.
WHY	The metrics we gate on are under our control and work offline. The vendor gives rich debugging at no extra code cost.
TRADE-OFF	SQLite fits a single instance only; in production it becomes an OpenTelemetry exporter.

ADR-10 Synchronous endpoints for now

DECISION	Plain request and response; planning takes several model calls within one request.
ALTERNATIVES	A job queue with progress streamed over server-sent events.
WHY	Simplest for a demo, and easy to reason about.
TRADE-OFF	Long requests hit platform timeouts and block a worker. This is the first thing I would change for production.

ADR-11 Split hosting: Vercel for the UI, a container for the API

DECISION	Next.js on Vercel; the Python API as a Docker container on Render, Railway or Fly.io.
ALTERNATIVES	Everything on Vercel serverless functions; everything in one container.
WHY	The embedding model, long agent runs and a writable telemetry store do not fit serverless limits.
TRADE-OFF	Two deployments and CORS configuration.

13 · BEYOND THE DEMO

Path to production

AREA	DEMO TODAY	PRODUCTION CHANGE	TRIGGER
Long-running work	Synchronous requests	Queue (for example Redis + workers), job ids, progress over SSE	p95 plan latency or platform timeouts
Knowledge base	7 hand-written files	Ingest official BAFA and KfW documents with a valid-from date per chunk; answer by the rules in force on the application date	Before any customer use
Retrieval quality	Hybrid + RRF	Add a reranker; evaluate on a larger labelled set	hit@5 below target on real questions

AREA	DEMO TODAY	PRODUCTION CHANGE	TRIGGER
Cost	Per-run budget, text-first, cache	Prompt caching for long system prompts, Haiku for extraction where accuracy allows, batch API for evals	Cost per analysis trend
Telemetry	SQLite + LangSmith	OpenTelemetry export, dashboards, alerts on p95, error rate, fallback rate and spend	More than one instance
Security	Regex PII redaction, no auth	Auth and tenant isolation, EU data residency review, a stronger PII detector, audit log	First external user
Evals	8 certificates, 14 questions	Grow from human corrections; second independent judge; regression set per customer document type	Every prompt or model change
Multimodal	Certificates	Floor plans and photos for wall and window areas, replacing the rough area factors	Cost accuracy versus site visits

Known limitations, stated plainly

- Cost and saving factors are rough averages; real estimates need geometry or a site visit.
- Heat pump sizing uses full-load hours, not a DIN EN 12831 heat load calculation.
- Funding rules are simplified from public summaries of the July 2026 reform and must be checked against BAFA and KfW.
- The rule-based extractor only handles the sample layout; real layouts rely on the AI path.
- All sample certificates are fictional.

14 · FOR THE INTERVIEW

Two-minute architecture pitch

> "I built a small copilot that goes from an energy certificate to a cited retrofit plan, and I designed it around trust. The model reads the document and decides which measures make sense, but code calculates every euro, grant and tonne of CO_2, and the agent can only cite sources it actually retrieved.

> Every feature has three levels of fallback: retries, a cheaper model, and a non-AI path, plus a cost cap per request. Every step is traced with model, tokens, cost and latency, and five eval suites score extraction, retrieval, answers and planning, with thresholds that fail CI.

> It's a modular monolith on purpose. The boundaries are where I'd split it later, starting with moving extraction and planning onto a queue. The shared LLM client, tracer and eval harness are the parts I'd want every squad to reuse."

Senior questions to expect

- How would you know a new model is better? Run all suites on both and compare per metric; ship behind a flag; watch fallback and correction rates.
- Where does this break at 100× load? Synchronous planning and SQLite telemetry. Queue and OpenTelemetry first.
- What would you build first at Wallround? An evaluation set from the documents the team already processes by hand, before any new feature.
- Why not let the model do the maths? Auditability and repeatability; see ADR-2.

Questions to ask them

- Which AI capability is closest to production today, and what is blocking it?
- How do you measure the accuracy of cost forecasts against final project costs?
- Which documents does the team still read by hand?
- What does the product stack look like, and how much of this role touches it?